

设备管理平台开放服务（钉钉）

一、文档概念

1.1 基本概念

1、应用

应用是指借助设备管理平台提供给第三方开发者使用的开放服务，开发者根据自己需求和场景创建应用。

2、回调

回调是指平台将一些数据主动发送给第三方开发者的一种行为，第三方开发者需要根据平台的接口规范，开发相应的回调接口。

目前只提供设备、录音通过实时回调给开发者接口，具体参数配置请查看接口文档。

二、接口说明

2.1 接口概述

平台注册的第三方平台，需要获得以下信息：

接入服务HOST

生产：<https://ecard-api-141770.eapps.dingtalkcloud.com/iot-cloud-open/v2>

名词说明

1. <version>代表api版本，目前最新版本为v2.0，最新地址为<https://ecard-api-141770.eapps.dingtalkcloud.com/iot-cloud-open/v2>

请求说明

1. 接口出参都是 JSON 格式
2. 时间单位“yyyy-MM-dd HH:mm:ss”或“yyyy-MM-dd”，如“2020-01-20 01:23:23”，“2020-01-20”。没有时间为空字符串
3. POST请求参数包括在body中（请求头：application/json）

返回值说明

1. 接口返回： 所有接口返回的数据， 都包含三个字段， 即 result、 code、 msg
2. result表示处理结果（ 1 成功， 0 失败）
3. code表示业务状态码, 1000为成功, 1001为通用失败
4. msg表示业务提示信息， 成功返回“success”
5. data表示结果集， result为1时返回结果集

成功返回示例：

✓ 复制代码

```
{
  "result": 1,
  "code": 1000,
  "msg": "success",
  "data": {
    "id": 1
  }
}
```

错误返回示例：

✓ 复制代码

```
{
  "result": 0,
  "code": 1002,
  "msg": "非法参数"
}
```

2.2 返回码说明

返回码	说明
1000	成功
1001	失败
1002	非法参数

1003	参数缺失
1004	设备离线
1005	设备正在录音/设备未开始录音
1006	设备GPS定位已开启/设备GPS定位未开启
1007	鉴权失败
1008	认证失败
1009	录音开启失败
10010	录音停止失败
10011	GPS开启失败
10012	GPS停止失败
10013	当前无权限操作设备
10014	开启录音响应超时(tcp连接不存在)/关闭录音响应超时(tcp连接不存在)
10015	开发者账号信息不存在
10016	当前设备号不支持此操作
10017	任务重复
10018	任务不存在
10019	当前并发请求过多，请稍候重试

2.3 鉴权说明

请求地址

<HOST>/auth/access_token

请求方式

POST

使用场景

获取token后，通过HTTPS请求调用业务接口，将token作为**Authorization**的值放入Headers进行校验。

接口说明

- 1、云端服务器需要鉴定用户是否有权调用业务接口，token为接口调用凭证，是接口请求的基本参数
- 2、timestamp的有效期为10s
- 3、token有效期为24小时，过期后需再次获取；建议定时更新token，比如每24小时更新一次
- 4、POST请求参数包括在body中（请求头：application/json）

请求参数

参数名	是否必须	类型	描述	说明
appld	必须	字符串	应用ID	无
timestamp	必须	数值	时间戳	UNIX时间戳(秒) 时间戳有效期 10s
sign	必须	字符串	签名	MD5-32(appld+timestamp+appSecret)，MD5加密，32位小写

∨

复制代码

```
{
  "timestamp": 1592445306,
  "appId": "123456",
  "sign": "ssdfsdf222cfcf02degrg"
}
```

返回参数

参数名	类型	描述
accessToken	string	鉴权Token
expiredTime	时间戳	UNIX时间戳(秒)

[复制代码](#)

```
{
  "result": 1,
  "msg": "成功",
  "code": 1000,
  "data": {
    "accessToken": "eff18f4a-1528-4eaa-baa8-a71b1988216b",
    "expiredTime": 1606221911
  }
}
```

访问接口服务

将accessToken放入对应业务接口Headers的**Authorization**中

2.4 加密说明

录音文件地址加密机制说明

概述

在 IOT 平台设备录音文件的访问/回调接口中，系统提供录音文件地址的可选加密机制，以保障文件访问安全性，默认为关闭。该加密功能通过配置项控制，当加密开关开启时，返回的文件地址将进行加密处理；关闭时，则以明文形式返回。

如果需要开启或关闭加密机制，请联系管理员

加密算法说明（AES-GCM-128）

- 算法名称：AES（Advanced Encryption Standard）
- 模式：GCM（Galois/Counter Mode）
- 密钥长度：128-bit（16 字节）
- IV**（Nonce）长度**：12 字节
- Tag 长度：16 字节

- **密文格式：** [Nonce (12字节)] + [Ciphertext + Tag]，整体进行 Base64 编码后返回

示例结构：



复制代码

Base64(Nonce + Ciphertext + AuthTag) → 字符串

加密端处理逻辑（服务端）

1. 生成 12 字节随机 Nonce
2. 使用预设密钥进行 AES-GCM 加密
3. 拼接 Nonce、Ciphertext 与 AuthTag
4. 对整个数据进行 Base64 编码后作为地址字段返回

Go 示例：



复制代码

```
ciphertext := aesGCM.Seal(nil, nonce, plaintext, nil)
encoded := base64.StdEncoding.EncodeToString(append(nonce,
ciphertext...))
```

解密端处理逻辑（客户端）

1. Base64 解码获得原始数据
2. 提取前 12 字节为 Nonce
3. 提取剩余部分为 Ciphertext + Tag
4. 使用相同密钥进行 AES-GCM 解密

Go 示例：



复制代码

```
nonceSize := aesGCM.NonceSize()
nonce := crypted[:nonceSize]
ciphertext := crypted[nonceSize:]
plaintext, err := aesGCM.Open(nil, nonce, ciphertext, nil)
```

三、接口请求

调用以下接口时，需要在请求 Headers 中添加参数 **Authorization**，其值为 **accessToken**。

3.1 设备

1.获取全部设备

请求地址

<HOST>/device/query_all

请求方式

POST

接口说明

获取全部设备数据

请求参数

参数名	是否必须	类型	描述	说明
page	必须	数值	分页页码	从1开始，默认为1
size	必须	数值	每页数量	最大100
deviceNo	非必须	字符串	设备号	支持单个设备号查询，不传或为空则查询所有设备信息，不支持多个设备号查询。
deviceType	非必须	字符串	设备类型	传入具体的设备类型查询该类型的所有设备信息，例如SX401
deviceGroupName	非必须	字符串	设备分组名称	传入具体的设备分组名称查询分组下所有的设备

				信息，只支持单个分组信息
remainPowerMin	非必须	数值	最小设备电量	单位百分比，筛选电量大于等于此电量的设备
remainPowerMax	非必须	数值	最大设备电量	单位百分比，筛选电量小于等于此电量的设备

请求参数示例

▼复制代码

```
{
  "page": 1,
  "Size": 100
}
```

返回参数

参数名	类型	描述
rows	<Device>集合	设备列表
total	数值	总条数
pages	数值	总页数
pageNum	数值	当前页

<Device>说明：

参数名	类型	描述
deviceNo	字符串	设备号
onlineStatus	数值	设备状态 0 离线 1在线
lastRecordTime	字符串	最后录音时间

lastOnlineTime	字符串	最后在线时间
joinTime	字符串	设备加入时间
recordStatus	数值	录音状态 0 未录音 1正在录音
gpsStatus	数值	GPS状态 0 关闭 1开启
firmwareVersion	字符串	固件版本号
remainPower	数值	电量
remainStorageSize	数值	剩余存储空间MB
pendingFileNum	数值	待上传文件数
msgType	数值	消息类型 0-心跳 1-开始录音 2-结束录音 3-设备在线 4-设备离线 5-接口请求 6-开机 7-录音上传 8-上传结束 9-关机
reportTime	时间	上报时间
iccid	字符串	ICCID
audioVolume	数值	设备音量 0-5 整数
allowPlay	数值	是否允许播报 0不允许 1允许
rsi	数值	信号值 0-31 （信号弱到强）
deviceType	字符串	设备型号
chargeStatus	数值	充电状态， 0-未充电 1-充电中
lastOfflineTime	字符串	上次离线时间


```

        "joinTime": "2020-10-01 09:23:23",
        "recordStatus": 0,
        "gpsStatus": 0,
        "firmwareVersion": "1.0.0",
        "remainPower": 60,
        "remainStorageSize": 1024,
        "pendingFileNum": 0,
        "msyType": 5,
        "reportTime": "2020-10-01 09:23:23",
        "iccId": "xxx",
        "audioVolume": 3,
        "allowPlay": 1,
        "rssi": 28,
        "deviceType": "SG301",
        "chargeStatus": 0,
        "lastOfflineTime": "",
        "recordMode": 0,
        "realTimeMode": 0,
        "deviceStatus": 1,
        "trafficExpirationDate": "2025-09",
        "deviceGroupName": "测试分组",
        "bluetoothStatus": 1,
        "employeeId": 3,
        "employeeName": "xxxxx",
        "departmentId": 9823,
        "departmentName": "xxxxx",
        "storeId": 2983,
        "storeName": "xxxxx"
    }
],
"total": 542,
"pages": 6,
"pageNum": 1
}

```

2.开启GPS

请求地址

<HOST>/device/gps/start

请求方式

POST

接口说明

开启设备GPS定位；

请求参数

参数名	是否必须	类型	描述	说明
deviceNo	必须	字符串	设备号	仅支持单个设备
orderNo	非必须	字符串	工单号	最大长度100字符

请求参数示例

∨

复制代码

```
{  "deviceNo": "xxxxx",  "orderNo": "xxxxx",}
```

返回参数

参数名	类型	描述

返回数据示例

∨

复制代码

```
{  "result": 1,  "msg": "success",  "code": 1000}
```

3.关闭GPS

请求地址

<HOST>/device/gps/stop

请求方式

POST

接口说明

关闭设备GPS定位;

请求参数

参数名	是否必须	类型	描述	说明
deviceNo	必须	字符串	设备号	仅支持单个设备

请求参数示例

▼

复制代码

```
{  "deviceNo":  "xxxxx"}
```

返回参数

参数名	类型	描述

返回数据示例

▼

复制代码

```
{  "result":  1,  "msg":  "success",  "code":  1000}
```

4.获取设备当前定位

请求地址

<HOST>/device/gps/curr_location

请求方式

POST

接口说明

1. 获取设备当前GPS定位，定位信息通过GPS数据回调接口返回

请求参数

参数名	是否必须	类型	描述	说明
deviceNo	必须	字符串	设备号	仅支持单个设备

请求参数示例

▼	复制代码
{ "deviceNo": "xxxxx" }	

返回数据示例

▼	复制代码
{ "result": 1, "msg": "success", "code": 1000 }	

5. 设备设置

请求地址

<HOST>/device/setting

请求方式

POST

接口说明

1. 修改指定设备配置

请求参数

参数名	是否必须	类型	描述	说明
deviceNos	必须	字符串数组	设备号组	支持多个设备
allowClip	必须	数值	是否允许拆分	0-不允许 1-允许
clipSeconds	必须	数值	拆分时长	单位：秒（最少设置300s）
failSeconds	非必须	数值	允许上传时长	单位：秒
allowPlay	非必须	数值	是否允许播报	0-不允许 1-允许

[复制代码](#)

```
{  "result": 1,  "msg": "success",  "code": 1000,  "data": {    "failed": ["xxxx","xxxx"],    "successful":["xxxx","xxxx"]  } }
```

6. 设备绑定员工

请求地址

<HOST>/device/batch-assign-employees

请求方式

POST

接口说明

1. 支持单个或批量为设备绑定员工信息
2. 当调用获取全部设备接口时，对同一设备，仅返回最新一条绑定成功的员工信息
3. 单次绑定支持不超过500条

请求参数

参数名	是否必须	类型	描述	说明
binds	必须	对象数组	绑定记录组	支持多个
deviceNo	必须	字符串	设备ID	设备唯一
employeeId	必须	数值	员工ID	员工唯一（填），传
employeeName	非必须	字符串	员工名称	绑定时的
departmentId	非必须	数值	部门ID	绑定时的
departmentName	非必须	字符串	部门名称	员工所属
storeId	非必须	数值	门店ID	绑定时的
storeName	非必须	字符串	门店名称	员工所属

operatorId	非必须	数值	操作人ID	执行绑定
------------	-----	----	-------	------

请求参数示例

▼复制代码

```
{
  "binds": [
    {
      "deviceNo": "xxxx",           // 必填：设备唯一标识
      "employeeId": 5001,          // 必填：员工ID
      "employeeName": "张三",      // 非必填：员工姓名（可为 null）
      "departmentId": 101,         // 非必填：部门ID（可为 null）
      "storeId": 201              // 非必填：门店ID（可为 null）
    },
    {
      "deviceNo": "DEV-002",
      "employeeId": 5002,
      "employeeName": "李四"
      // employeeName, departmentId, storeId和operatorId 可省略或传 null
    }
  ]
  // 支持多条，建议单次不超过 500 条，避免超时
}
```

返回参数

参数名	类型	描述
data	bindsData对象	设备绑定员工结果

<bindsData>说明：

参数名	类型	描述
total	数值	总数
success	数值	绑定成功记录数

failed	数值	绑定失败记录数。失败原因：deviceNo不存在
failedList	数组	失败设备号清单（若有）

返回数据示例

 复制代码

```
{
  "result": 1,
  "msg": "success",
  "code": 1000,
  "data": {
    "total": 3,
    "success": 2,
    "failed": 1,
    "failedList": [
      "xxxx"
    ]
  }
}
```

3.2 录音

1. 开启录音

请求地址

<HOST>/audio/start

请求方式

POST

接口说明

1. 开启设备录音;
2. 因设备网络情况而定，请求响应时间最大为32s
3. 如果设备开启了实时流录音，开启录音收到的录音数据回调中会出现wsUrl
(ws://{server_address}/stream/{stream_id})，拼接token后可得到完整的ws订阅地址

请求参数

参数名	是否必须	类型	描述	说明
deviceNo	必须	字符串	设备号	
orderNo	非必须	字符串	工单号	最大长度100字符

请求参数示例

▼	复制代码
{ "deviceNo": "xxxxx", "orderNo": "xxxxx"}	

返回参数

参数名	类型	描述

返回状态码: 1001、1002、1009、10013、10014

返回数据示例

▼	复制代码
{ "result": 1, "msg": "success", "code": 1000}	

2.停止录音

请求地址

<HOST>/audio/stop

请求方式

POST

接口说明

1. 停止设备录音;
2. 因设备网络情况而定，请求响应时间最大为32s

请求参数

参数名	是否必须	类型	描述	说明
deviceNo	必须	字符串	设备号	

请求参数示例

▼

复制代码

```
{  "deviceNo": "xxxxx"}
```

返回参数

参数名	类型	描述

返回状态码: 1001、1002、1005、10010、10013、10014

返回数据示例

▼

复制代码

```
{  "result": 1,  "msg": "success",  "code": 1000}
```

3.获取录音记录

请求地址

<HOST>/audio/query_all

请求方式

POST

接口说明

1. 获取录音记录
2. 当 IOT 录音文件的加密开关关闭时，接口返回的数据中包含的文件地址为明文格式；当加密开关开启时，返回的文件地址将以加密形式传输，具体加密形式参考接口2.4加密说明。

请求参数

参数名	是否必须	类型	描述	说明
page	必须	数值	分页页码	从1开始，默认为1
size	必须	数值	每页数量	最大100
deviceNo	非必须	字符串	设备号	
orderNo	非必须	字符串	工单号	
startTime	必须	字符串	查询开始时间	yyyy-MM-dd HH:mm:ss
stopTime	必须	字符串	查询结束时间	yyyy-MM-dd HH:mm:ss
uploadStatus	非必须	数值	上传状态	0-未上传 1-已上传

请求参数示例

▼	复制代码
<pre>{ "page": 1, "size": 100, "startTime": "2022-08-17 00:00:00", "stopTime": "2022-08-18 00:00:00"}</pre>	

返回参数

参数名	类型	描述
rows	<Audio>集合	设备列表
total	数值	总条数
pages	数值	总页数
pageNum	数值	当前页

参数名	类型	描述
deviceNo	字符串	设备号
eventId	字符串	录音ID
miniold	字符串	录音文件MD5校验码
orderNo	字符串	工单号
fileName	字符串	录音文件名称
fileSize	数值	录音文件大小 单位：B
startTime	字符串	录音开始时间
stopTime	字符串	录音结束时间
seconds	数值	录音时长，秒
fileUrl	字符串	录音文件
rightFileUrl	字符串	右声道录音文件
leftFileUrl	字符串	左声道录音文件
audioFileExpirationTime	字符串	录音文件（录音文件，左声道，右声道文件）链接过期时间。链接过期前五分钟会自动刷新，建议在过期前五分钟内获取更新后的录音链接
seq	数字	切片序号
endMark	数字	结束标识 0-本次切片不是最后一次切片 1-本次切片是最后一次切片
recType	数字	录音类型 0-音频 1-声纹

返回数据示例

[复制代码](#)

```
{  "result": 1,  "msg": "success",  "code": 1000,  "data": {    "rows": [      {        "deviceNo": "xxxxx",        "eventId": "xxxxx",        "minioId": "xxxxx",        "orderNo": "xxxxx",        "fileName": "a.mp3",        "fileSize": 1024,        "startTime": "2020-10-01 09:23:23",        "stopTime": "2020-10-01 09:23:23",        "seconds": 100,        "fileUrl": "http:///a.mp3",        "rightFileUrl": "http:///a.mp3",        "leftFileUrl": "http:///a.mp3",        "audioFileExpirationTime": "2020-10-01 09:23:23",        "seq": 1,        "endMark": 1      }    ],    "total": 542,    "pages": 6,    "pageNum": 1  }}
```

切片结果回调示例

[复制代码](#)

```
{  "oriTaskId": "",  // 调用方任务ID  "taskId": "",  // 任务ID  "fileUrl": "",  // 音频地址  "status": 0,  // 任务状态: 0: 待执行, 1: 执行中, 2: 执行失败, 3: 执行成功  "errMsg": "",  // 错误信息  "orderNo": ""}
```

4.提交录音切片任务

请求地址

<HOST>/audio/task/submit

请求方式

POST

接口说明

1. 创建录音切片任务，创建任务时间段要求不小于1分钟，最大支持时间段在24小时内的合并任务
2. 场景描述：创建任务，将某设备在开始时间和结束时间之内的录音拼接为一个录音文件

3. 如果系统中已存在具有相同设备号 + 开始时间 + 结束时间组合的切片任务，且该任务执行状态（status：3）成功，则不支持创建新的重复切片任务。执行失败状态（status：2）的任务不视为重复
4. 回调切片任务结果

请求参数

参数名	是否必须	类型	描述	说明
deviceNo	必须	字符串	设备号	支持单个设备
callbackUrl	非必须	字符串	回调地址	
oriTaskId	非必须	字符串	调用方分片任务ID	
orderNo	非必须	字符串	工单号	最大长度100字符
timeSlots	必须	数组	时间段列表	暂时只支持一个时间段（时间段需要大于等于1分钟）
timeSlots.startTime	必须	字符串	开始时间	格式: yyyy-MM-dd HH:mm:ss
timeSlots.endTime	必须	字符串	结束时间	格式: yyyy-MM-dd HH:mm:ss

请求参数示例

复制代码

```
{  "deviceNo": "xxxxxxxx",    "callbackUrl": "xxxxxxxx",  "oriTaskId": "xxxxxxxxxxxxxxxx",  "orderNo": "xxxxxxxx",  "timeSlots":  [    {      "startTime": "2022-08-17 10:00:00",      "endTime": "2022-08-17 11:00:00"    }  ]}
```

成功

参数名	类型	描述	说明
data	TaskData对象		创建切片任务成功时，返回code：1000

<TaskData>

参数名	类型	描述	说明
oriTaskId	字符串	调用方任务ID	
taskId	字符串	系统任务ID	

▽

 复制代码

```
{  "result": 1,  "msg": "success",  "code": 1000,  "data": {    "oriTaskId": "xxxxxxxxxxxx",    "taskId": "xxxxxxx"  }}
```

失败示例

▽

 复制代码

```
{  "result": 0,  "msg": "当前设备号不支持此操作",  "code": 10016,  "data": {}}
```

或者

```
{  "result": 0,  "code": 10018,  "msg": "任务失败,切分时间范围至少为一分钟"}
```

切片结果回调示例（任务创建后排队处理完成，向callbackUrl回调切片任务结果）

▽

 复制代码

```
{  "oriTaskId": "",  // 调用方任务ID  "taskId": "",  // 任务ID
```

```
"fileUrl": "", // 音频地址
"status": 0, // 任务状态: 0: 待执行, 1: 执行中, 2: 执行失败, 3: 执行成功
"errMsg": "", // 错误信息
"orderNo": ""
}
```

5.查询录音切片任务信息

请求地址

<HOST>/audio/task/query/{taskId}

请求方式

GET

接口说明

- 1. 查询任务状态信息
- 2. 切片结果为“执行失败”（status：2）有以下几种原因：
 - a. 当前时间段内无录音
 - b. 当前时间段原始录音文件时长总和不足1分钟
 - c. 当前时间段有录音文件还未上传完成
- 3. 当 IOT 录音文件的加密开关关闭时，接口返回的数据中包含的文件地址为明文格式；当加密开关开启时，返回的文件地址将以加密形式传输，具体加密形式参考接口**2.4加密说明**。

返回参数

参数名	类型	描述
data	TaskData对象	

<data>切片任务结果

参数名	类型	描述
oriTaskId	字符串	调用方任务ID
taskId	字符串	系统任务ID

orderNo	字符串	工单号
fileUrl	字符串	音频地址
status	数值	任务状态：1：待执行，2：执行失败，3：执行成功，4：执行中
errMsg	字符串	错误信息

返回数据示例

▼复制代码

```
{  "result": 1,  "msg": "success",  "code": 1000,  "data": {    "oriTaskId": "xxxxxxxxxxx",    "taskId": "xxxxxxxx",    "orderNo": "xxxxxxxx",    "fileUrl": "xxxxxxxxxxx",    "status": 2,    "errMsg": "文件还未上传完成"  }}
```

3.3 声纹

1.创建声纹

请求地址

<HOST>/voice-print/create

请求方式

POST

接口说明

1. 创建声纹录制任务并下发到对应设备，用户面对设备录制声纹素材，上传后生成声纹记录

请求参数

参数名	是否必须	类型	描述	说明
deviceNo	必须	字符串	设备号	
callbackUrl	必须	字符串	回调地址	

请求参数示例

∨	复制代码
<pre>{ "deviceNo": "xxxxx", "callbackUrl": "xxxxx"}</pre>	

返回参数

参数名	类型	描述
voicePrintId	字符串	声纹ID

返回数据示例

∨	复制代码
<pre>{ "result": 1, "msg": "success", "code": 1000, "data": { "voicePrintId": "xxxxxxxxxxxxxxxxxxxx" }}</pre>	

声纹回调

∨	复制代码
<pre>{ "code": 1000, "result": 1, "msg": "", "data": { "voicePrintId": "", // 声纹id "voicePrintUrl": "" // 声纹url地址 } }</pre>	

2.修改声纹

请求地址

<HOST>/voice-print/update

请求方式

POST

接口说明

1. 修改已录制的声纹素材，请求时设备号和声纹ID必填

请求参数

参数名	是否必须	类型	描述	说明
deviceNo	必须	字符串	设备号	
callbackUrl	必须	字符串	回调地址	
voicePrintId	必须	字符串	声纹ID	

请求参数示例

∨	复制代码
<pre>{ "deviceNo": "xxxxx", "callbackUrl": "xxxxx", "voicePrintId": "xxxxxxxx"}</pre>	

返回参数

参数名	类型	描述
voicePrintId	字符串	声纹ID

返回数据示例

∨	复制代码
<pre>{ "result": 1, "msg": "success", "code": 1000, "data": { "voicePrintId": "xxxxxxxxxxxxxxxxxxxxx" } }</pre>	

3.获取声纹列表

请求地址

<HOST>/voice-print/list

请求方式

POST

接口说明

1. 获取声纹列表

请求参数

参数名	是否必须	类型	描述	说明
page	必须	数值	分页页码	从1开始，默认为1
size	必须	数值	每页数量	最大100
deviceNo	非必须	字符串	设备号	
voicePrintId	非必须	字符串	声纹ID	
startTime	非必须	字符串	查询开始时间	yyyy-MM-dd HH:mm:ss
stopTime	非必须	字符串	查询结束时间	yyyy-MM-dd HH:mm:ss

请求参数示例

▼

 复制代码

```
{  "page": 1,  "size": 100}
```

返回参数

参数名	类型	描述
rows	<VoicePrint>集合	声纹信息列表

4.删除声纹

请求地址

<HOST>/voice-print/delete

请求方式

POST

接口说明

1. 删除声纹信息

请求参数

参数名	是否必须	类型	描述	说明
voicePrintId	必须	字符串	声纹ID	

请求参数示例

▼

复制代码

```
{  "voicePrintId": "xxxxxxxxx"}
```

返回参数

参数名	类型	描述

返回参数示例

▼

复制代码

```
{  "result": 1,    "msg": "success",    "code": 1000}
```

四、服务回调

回调逻辑说明

设备状态回调：

触发方式1：录音工牌向服务器发起设备状态心跳时触发网关回调设备状态至第三方服务，目前默认硬件心跳时间30分钟一次，回调第三方服务频率同心跳频率。

触发方式2：录音工牌开启或关闭录音（接口或按键），实时调起网关回调第三方服务。

录音数据回调：录音工牌关闭录音并完成录音文件上传触发网关回调对应设备录音记录和录音地址至第三方服务。

GPS数据回调：GPS开启后，硬件目前默认5S获取一次坐标。并随心跳频率回传GPS数据组至网关，网关实时回调GPS数据至第三方服务。

4.1 设备状态

回调说明

1. 用于企业内设备发生变化后(设备从离线变为在线，在线变为离线，开启/停止录音)回调第三方应用服务器地址
2. 管理员需在企业后台管理系统开发者设置中设置回调地址Url
3. 回调地址Url需支持POST协议

JSON数据包说明

参数名	类型	描述
callback	字符串	固定为DEVICE_INFORM
data	<Device>列表	设备数据

<Device>说明：

参数名	类型	描述
deviceNo	字符串	设备号
onlineStatus	数值	设备状态 0 离线 1在线
lastOnlineTime	字符串	最后在线时间

joinTime	时间	设备加入时间
recordStatus	数值	录音状态 0 未录音 1正在录音
gpsStatus	数值	GPS状态 0 关闭 1开启
firmwareVersion	字符串	固件版本号
remainPower	数值	电量
remainStorageSize	数值	剩余存储空间MB
pendingFileNum	数值	待上传文件数
msgType	数值	消息类型 0-心跳 1-开始录音 2-结束录音 3-设备在线 4-设备离线 5-接口请求 6-开机 7-录音上传 8-上传结束 9-关机 10-开始下载固件 11-固件下载失败 12-固件下载完成 13-远程开始录音 14-远程结束录音 15-蓝牙开启录音 16-蓝牙关闭录音 17-切分开启录音 18-切分关闭录音 19-自动开启录音 20-超时关闭录音
reportTime	时间	上报时间
iccid	字符串	ICCID
audioVolume	数值	设备音量 0-5 0最小5最大
allowPlay	数值	是否允许播报 0-不允许 1-允许
rsi	数值	信号值 0-31 （信号弱到强）
deviceType	字符串	设备型号


```
"gpsStatus": 0,
"firmwareVersion": "1.0.0",
"remainPower": 60,
"remainStorageSize": 1024,
"pendingFileNum": 0,
"msgType": 0,
"reportTime": "2022-11-29 15:04:05",
"iccId": "xxxx",
"audioVolume": 3,
"playStatus": 1,
"rssi": 28,
"deviceType": "SG301",
"chargeStatus": 0,
"lastOfflineTime": "",
"recordMode": 0,
"realTimeMode": 0,
"deviceStatus": 1,
"trafficExpirationDate": "2025-09",
"bluetoothStatus": 1,
"employeeId": 5001,
"employeeName": "张三",
"departmentId": 101,
"storeId": 201
}
]
```

4.2 录音数据

回调格式

ContentType: application/json

请求方式

POST

回调说明

1. 用于企业内设备发生录音后回调第三方应用服务器地址
2. 管理员需在企业后台管理系统开发者设置中设置回调地址Url
3. 回调地址Url需支持POST协议
4. 通话的录音回调会有少许延迟

5. 录音文件中的工单号由服务端生成，规则：设备号-录音开始时间时间戳(如果配置了切片则是第一个切片开始时间时间戳)
6. 当 IOT 录音文件的加密开关关闭时，接口返回的数据中包含的文件地址为明文格式；当加密开关开启时，返回的文件地址将以加密形式传输，具体加密形式参考接口**2.4加密说明**。

JSON数据包说明

参数名	类型	描述
callback	字符串	固定为AUDIO_INFORM
data	<AUDIO>数组	录音数据

参数名	类型	描述
deviceNo	字符串	设备号
eventId	字符串	录音ID
miniold	字符串	录音文件MD5校验码
orderNo	字符串	工单号
fileName	字符串	录音文件名称
fileSize	数值	录音文件大小 单位：B
startTime	字符串	录音开始时间
stopTime	字符串	录音结束时间
seconds	数值	录音时长，秒
fileUrl	字符串	录音文件
rightFileUrl	字符串	右声道录音文件
leftFileUrl	字符串	左声道录音文件

POST

回调说明

1. 用于GPS定位地址回调第三方应用服务器地址
2. 管理员需在企业后台管理系统开发者设置中设置回调地址Url
3. 回调地址Url需支持POST协议
4. 回调数据为数组，可能有一条或多条数据

JSON数据包说明

参数名	类型	描述
callback	字符串	固定为GPS_INFORM
data	<Gps>列表	GPS数据

<Gps>说明：

参数名	类型	描述
deviceNo	字符串	设备号
orderNo	字符串	工单号
receiveTime	字符串	接收时间
longitude	字符串	经度
latitude	字符串	纬度

<Lbs>说明：

参数名	类型	描述
cells	二维数组	基站信息
macs	二维数组	WIFI信息

JSON数据包示例

[复制代码](#)

```
{
  "callback": "GPS_INFORM",
  "data" : [{
    "deviceNo": "xxxxx",
    "orderNo": "xxxxx",
    "receiveTime": "2020-10-01 09:23:23",
    "longitude": "116.397128",
    "latitude": "39.916527",
    "type": 3,
    "lbsData": {
      "cells": [
        [
          460,
          11,
          28937,
          150216192,
          17,
          0,
          0,
          0,
          0,
          0
        ],
        [
          460,
          1,
          28937,
          150216192,
          0,
          0,
          0,
          0,
          0,
          0
        ]
      ]
    }
  ]
}
```



```
    0,  
    0,  
    0,  
    0  
  ],  
  [  
    460,  
    11,  
    28937,  
    27823473,  
    0,  
    0,  
    0,  
    0,  
    0,  
    0  
  ],  
  [  
    460,  
    11,  
    28937,  
    150216273,  
    0,  
    0,  
    0,  
    0,  
    0,  
    0  
  ],  
  [  
    460,  
    11,  
    28937,  
    182552601,  
    0,  
    0,  
    0,  
    0,  
    0,  
    0  
  ],  
],
```

```
[
  460,
  11,
  28937,
  180362650,
  0,
  0,
  0,
  0,
  0,
  0
],
[
  460,
  11,
  28937,
  26332990,
  0,
  0,
  0,
  0,
  0,
  0
]
],
"macs": [
  [
    "6877243807D2",
    -63
  ],
  [
    "480EEC64ADCE",
    -67
  ],
  [
    "38881E6EB518",
    -74
  ],
  [
    "7C992EBBA4C8",
    -82
  ]
]
```

```
    ],  
    [  
        "D076E7C45D9B",  
        -86  
    ],  
    [  
        "8E946A138C51",  
        -90  
    ],  
    [  
        "8E946A2392B3",  
        -93  
    ],  
    [  
        "D86D17CF142C",  
        -97  
    ]  
]  
}
```

4.4 GPS状态数据

回调格式

ContentType: application/json

请求方式

POST

回调说明

1. 用于企业内设备GPS状态发生变化后(设备GPS从关闭变为开启, 开启变为关闭)回调第三方应用服务器地址
2. 管理员需在企业后台管理系统开发者设置中设置回调地址Url
3. 回调地址Url需支持POST协议

JSON数据包说明

回调说明

1. 用于企业内设备出现异常后回调第三方应用服务器地址
2. 管理员需在企业后台管理系统开发者设置中设置回调地址Url
3. 回调地址Url需支持POST协议

JSON数据包说明

参数名	类型	描述
callback	字符串	固定为 DEVICE_EXCEPTION_IN FORM
data	<DeviceException>列表	设备异常数据

<DeviceException>说明：

参数名	类型	描述
deviceNo	字符串	设备号
firmwareVersion	字符串	固件版本号
msgType	数值	消息类型 0-未找到sd卡 1-远程升级访问失败 2-远程升级失败 3-sd卡写入失败 4-录音文件丢失 100-自定义错误
reportTime	时间	上报时间
description	字符串	异常描述

JSON数据包示例

▼

复制代码

{

```
"callback": "DEVICE_EXCEPTION_INFORM",
"data" : [{
  "deviceNo": "xxxxx",
  "firmwareVersion": "1.0.0",
  "msgType": 0,
  "reportTime": "2022-11-29 15:04:05",
  "description": "远程升级访问失败"
}]
}
```

4.6 WebSocket音频流服务

1.连接信息

基础URL格式

[复制代码](#)

```
ws://{server_address}/stream/{stream_id}?appToken={token}&appId=
{appId}&timestamp={timestamp}
```

参数说明

- `server_address` : 服务器地址和端口
- `stream_id` : 音频流ID, 由TCP服务端生成并返回
- `appToken` : 应用令牌, 用于验证客户端身份,MD5-32(appId+timestamp+appSecret), MD5加密, 32位小写
- `appId` :应用ID, 用于验证客户端身份
- `timestamp` :UNIX时间戳(秒), 实时获取token时时间戳有效期10s

示例

[复制代码](#)

```
wss://live-stream.dudutalk.com/stream/550e8400-e29b-41d4-a716-446655440000?  
appToken=your_token_here&appId=your_appId&timestamp=1748922071
```

2. 消息格式

2.1 服务端推送的音频帧格式

服务器端推送经过 Base64 加密的音频帧（每帧 40ms），本地解密后可获得 16kHz 采样率、16bit 位深的 Opus 编码并封装为 Ogg 格式的文件。为播放接收到的 Ogg 文件，建议根据文件头中的Ogg配置信息进行解析，以确保兼容性和正确解码。

解析二进制格式音频帧可参考以下示例（建议采用WebRtc协议解析音频流）：

下面是文件第一页的数据：

Address	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	Dump
00000000	4f	67	67	53	00	02	00	00	00	00	00	00	00	00	23	49	OggS.....#I
00000010	02	11	00	00	00	00	df	e2	0c	1f	01	13	4f	70	75	73	诶....Opus
00000020	48	65	61	64	01	01	38	01	80	bb	00	00	00	00	00	4f	Head..8.e?...C
00000030	67	67	53	00	00	00	00	00	00	00	00	00	00	23	49	02	ggS.....#I.
00000040	11	01	00	00	00	d4	3e	3f	20	03	ff	ff	fe	4f	70	75	?? . 隔pt
00000050	73	54	61	67	73	0b	00	00	00	6c	69	62	6f	70	75	73	sTags....libopus
00000060	20	31	2e	31	01	00	00	00	25	00	00	00	45	4e	43	4f	1.1....%...ENCO

- capture_pattern字段：值为 0x4f、0x67、0x67、0x53，对应字符 OggS，表示页起始标志；
- stream_structure_version字段：值为 0x00，表示版本号；
- header_type_flag字段：值为 0x02，表示逻辑比特流(bos)的第一页；
- granule_position字段：值为 0x0，媒体编码相关的参数信息，表示到本页为止逻辑流有0个采样；
- bitstream_serial_number字段：0x23、0x49、0x02、0x11，逻辑比特流序列号；
- page_sequence_number字段：0x00、0x00、0x00、0x00，页面序列号；
- CRC_checksum字段：0xdf、0xe2、0x0c、0x1f，32位CRC校验；
- number_page_segments字段：0x01，表示后面段(segment)的个数，本页有1个段(segment)；
- segment_table：前面指明了只有1个段，所以segment_table大小为1个字节，值为 0x13，表示这个段的大小为 0x13 个字节。
- 后面的0x13个字节就是段的内容(上图中浅蓝色背景的字节)：0x4f 开始到 0x00。

[复制代码](#)

```
{  
  "deviceNo": "DEVICE_TEST_0174",    // 设备编号  
  "audioName": "54c892cb-7ffe-43c1-b903-  
99b45b8a7278HSiVnOPSiATRv3Sq.ogg",  // 音频文件名
```

```

"streamId": "48f7213e-ae41-436a-9a8a-eb1b344b31b5", // 流ID
"startTime": 1747892821045, // 开始时间戳（毫秒）
"stopTime": 1747892821085, // 结束时间戳（毫秒）
"timestamp": 1747892821045, // 当前时间戳（毫秒）
"status": 1, // 状态码
"dataLen": 1280, // 数据长度（base64解码后）
"data":
"Mmn3iOEa24bIt/xyh9xjMMsIgeGQwZFLAhvOcvejXzTu2EUvHBd4BDVTShtKU7ZbhTZq7m2
S5hLgIvnT7R49y522bFe0ZAG+NNA82rziQmNZhyZJnB0HbsY81EypnkWDh38aN0BvdJidyxw
eQkNlZoW4rX9VIyOSdGSuzd7PjqG1UbD0CcijfaVRgDZEcBnyMtLQBfYmOR5npby0drqhXK/
ps2208B+REyYe4+Mmv0D0rPaoHJW/HTJMMHSX7Aunf7Pjp2RaT4lp6jcMBvIPqySJ+TvxRzF
hjk019zcEoQViixSZ6xc1ofpXNSYDoS5Q0tBDfBUHmWtTZkpQZFhZRMxpWMTBRi0Ri3J7ad
vr1jmE/6QuxVW4fw0nRq9hzF5uLSHIAgGadtV+5xutL47h0q9yWu5qurQjIvwRVMA7U6VzyX
L4qfj81SeN0pNZfHmegDkWP9eSSsd1CPeRCUY0wYSy+2BaFrFrqSEAZp/VJ2q48gLtK5giV
el+qYo9QjiPhBt2izPpm8uN5SLQLp45/MTYiMhsN+WRTDKceVq7JYzwNtr5p2msKY5BLfVuB
Gyou1N9sAPPq//eNDSHzL9UMkmwFhkHenCYBWssMtLnVj3V15uMM0aAUUnKdcFFE3lio2bWBV
vZvLZVDeV/sr+kA6XrKz/IC61KplqrnNvNTctWuRUCMutBBgcbyTnDg7ymDY1hrdYNvruwes
B/ec0dzH00kzHVE0YhIXxv6jTzpf6oTqQkKSIKayOgM4gePx/gm7Zki5+XxJk/WZvr5cN42w
Y4pCB90RcVhp4T3ImvBYIZDcy+P8+ww4VxNeAciw9g3wQ1/UgSxr+pwAWbp1GfYB8ZVNbPDP
PDpE02nUAF2CRZyuCBbjBP9+XTXxAiJivfusnyTLRCZuQ7oIH6s0JkOEDdH0t0CX2rDZoQwi
fkKD3+FY05SL0bW9hNRITiYEHCPzr4c0sg5dQ4hkMMe0kOCD5CWYRpqqxyXs1dQ9Qhemzgyb
ZFFy+mnuR5aJUfMXQXjCsrJzQpzN2/HUzkxpWj3s+4V+MEo9gBd+3poId05j+dqkHh0SddPX
fIppq0ppq5mxB5p1NytMIosuzTqG/Yxnn3HKp6oo3glD0gLSgWthG4xIzKGMscglMde38u0Vt
xX2hgauJG8TQYLfyk1XM9tnE0bR2esZ09kRmzh5ZECJDmoMacZYqAYlqKae0iRSWAwL+KU+o
rZZZiMj+B9UwJtDWNyXSc2HenVfuBfhMpw4KGhRybeXY3xAm1i9NBd7EZQ0++L4Qf0e6jub0
27tFaB3eaWoaggD04mM163lmfKbcVGooAGtCTW5KKqPsxS1S5+kAhqqm4wRVuUzLvIK2v0Dq
eNsTZvHP1lXeoAovH4XDbvDATOYdgnoy0hn7Ay464y6zrhv9h/RQ/uRtC9WrPnBt2CcMtEDv
SH04W+/R4ILPBPBLwoJZZ7cudibKHEzAGDFfJH61f07mhWWpi5ZQeqgI+LMuRrAaa20J0x1Z
pFW0Br0LVSReq72Bqr3BqhK7NuMKuU1qCDSohtCp740c1JJgGSm3Dc3B0cR61VHTDZx/iT9x
U1Sn1spRP8TbIzYZkvPZXGyIemxv75RMb/9ZEvC7zATqdJ3UPZr81Ju2iXfbgxZW/3fSyXs9
j+dU2xIUwZ+n1YkUtmbtkSHKVa3P83LpJVWuPGNkmR1kX+Ob0juU=" // Base64
编码的音频数据
}

```

2.2 状态码说明

[复制代码](#)

- 0: 开始
- 1: 发送中
- 2: 结束

3. 连接流程

1. 建立连接

- a. 客户端通过WebSocket URL连接服务器
- b. URL必须包含有效的appToken
- c. 服务器验证appToken并升级连接

2. 接收数据

- a. 服务器会持续推送音频帧数据
- b. 每帧数据包含40ms的音频内容
- c. 数据以Base64编码格式发送

3. 连接终止

- a. 当收到status=2的帧时，表示音频流结束
- b. 服务器会发送关闭消息
- c. WebSocket连接将被正常关闭

4. 实时流解析方式推荐

- a. 建议将解码后的二进制数据通过WebRTC等协议机制进行实时音频流处理，以确保低延迟传输和高兼容性
- b. 不支持直接嵌入Web浏览器播放

4. 错误处理

4.1 连接错误

- 401: 无效的appToken
- 400: 缺少stream_id参数

4.2 运行时错误

- WebSocket连接异常关闭
- 数据解码错误
- 流不存在或已结束

5. 注意事项

1. 连接维护

- a. 客户端应该处理连接断开的情况
- b. 建议实现自动重连机制
- c. 处理WebSocket的错误事件

2. 数据处理

- a. 音频数据为Base64编码，使用前需解码
- b. 每帧音频数据固定为40ms
- c. 按照timestamp顺序处理数据

3. 性能考虑

- a. 及时处理接收到的数据
- b. 注意内存使用，避免数据堆积
- c. 保持稳定的网络连接

6. 示例代码

接收OPUS流并存储Ogg文件示例（Golang版）

✓ 播放Ogg文件

📄 复制代码

```
package main

import (
    "bytes"
    "encoding/base64"
    "encoding/json"
    "fmt"
```

```

"os"
"os/exec"
"path/filepath"

"github.com/gorilla/websocket"
"github.com/pion/rtp"
"github.com/pion/webrtc/v4/pkg/media/oggwriter"
"mccoy.space/g/ogg"
)

// DataResponse 与后端接口的 DataResponse 结构对齐（仅保留必需字段）
type DataResponse struct {
    Status int    `json:"status"`
    DataLen int    `json:"dataLen"`
    Data    string `json:"data"`
}

// StreamToOgg 从 WebSocket 实时 Ogg/Opus 流中提取音频帧，生成合法的
// Ogg/Opus 文件。
// 支持 fallback: 如果数据不是标准 Ogg 页，则视作单个 Opus 帧处理。
// 无时长限制：持续接收直到 WebSocket 断开。
// 参数：
//   - wsURL: WebSocket 流地址（必填，例如 "wss://live-
//     stream.example.com/stream/{id}"）。
//   - outDir: 输出目录（默认 "./out"）。
//   - baseName: 输出文件名基础（默认 "stream_<timestamp>"）。
//   - frameMS: 每帧毫秒时长（默认 40ms，用于 RTP 时间戳计算）。
// 返回：生成的 Ogg 文件路径（成功时），或错误。
func StreamToOgg(wsURL, outDir, baseName string, frameMS int) (string,
error) {
    if wsURL == "" {
        return "", fmt.Errorf("wsURL is required")
    }
    if outDir == "" {
        outDir = "./out"
    }
    if err := os.MkdirAll(outDir, 0o755); err != nil {
        return "", fmt.Errorf("mkdir %s: %w", outDir, err)
    }
    if baseName == "" {

```

```

        baseName = fmt.Sprintf("stream_%d", os.Getpid()) // 使用 PID 避免
文件名冲突（无限录制）
    }
    oggPath := filepath.Join(outDir, baseName+".ogg")

    // RTP 时间戳配置：Opus 采样率 48kHz，每帧步进 = 48000 * (frameMS /
1000)
    step := uint32(48000 * frameMS / 1000)
    var timestamp uint32 = 0

    // 连接 WebSocket
    conn, _, err := websocket.DefaultDialer.Dial(wsURL, nil)
    if err != nil {
        return "", fmt.Errorf("dial ws %s: %w", wsURL, err)
    }
    defer conn.Close()

    // 创建 Ogg 写入器（采样率 48kHz，单声道）
    writer, err := oggwriter.New(oggPath, 48000, 1)
    if err != nil {
        return "", fmt.Errorf("new oggwriter: %w", err)
    }
    defer writer.Close()

    fmt.Printf("开始无限录制 Ogg 流（直到 WSS 断开）: %s\n", oggPath)

    // 无限循环：直到 WebSocket 读取失败（断开）
    for {
        _, msg, err := conn.ReadMessage()
        if err != nil {
            fmt.Printf("WebSocket 断开（结束录制）: %v\n", err)
            break // 连接断开，退出循环
        }

        var resp DataResponse
        if err := json.Unmarshal(msg, &resp); err != nil {
            return "", fmt.Errorf("unmarshal json: %w", err)
        }

        if resp.Data != "" && resp.DataLen > 0 {
            // Base64 解码

```

```

payload, err := base64.StdEncoding.DecodeString(resp.Data)
if err != nil {
    return "", fmt.Errorf("base64 decode: %w", err)
}

// 优先解析为 Ogg 页
dec := ogg.NewDecoder(bytes.NewReader(payload))
page, parseErr := dec.Decode()
if parseErr == nil && len(page.Packets) > 0 {
    // 多包 Ogg 页: 逐包写入 RTP
    for _, pkt := range page.Packets {
        rtpPkt := &rtp.Packet{
            Header: rtp.Header{Version: 2, PayloadType:
111, Timestamp: timestamp, SSRC: 1},
            Payload: pkt,
        }
        if err := writer.WriteRTP(rtpPkt); err != nil {
            return "", fmt.Errorf("write rtp packet: %w",
err)
        }
        timestamp += step
    }
} else {
    // Fallback: 整块作为单个 Opus 帧
    rtpPkt := &rtp.Packet{
        Header: rtp.Header{Version: 2, PayloadType: 111,
Timestamp: timestamp, SSRC: 1},
        Payload: payload,
    }
    if err := writer.WriteRTP(rtpPkt); err != nil {
        return "", fmt.Errorf("write rtp fallback: %w", err)
    }
    timestamp += step
}
}

if resp.Status == 2 { // 流结束信号 (可选)
    fmt.Println("收到流结束信号 (Status=2)")
    break
}
}

```

```

    fmt.Printf("Ogg 文件保存完成: %s\n", oggPath)
    return oggPath, nil
}

// ConvertToMP3 可选: 使用 FFmpeg 将 Ogg 转换为 MP3 (需本地安装 FFmpeg)。
// 参数:
//   - oggPath: 输入 Ogg 文件路径。
//   - mp3Path: 输出 MP3 文件路径。
// 返回: 错误 (如果转换失败)。
func ConvertToMP3(oggPath, mp3Path string) error {
    cmd := exec.Command("ffmpeg", "-y", "-i", oggPath, "-codec:a",
"libmp3lame", "-b:a", "128k", mp3Path)
    output, err := cmd.CombinedOutput()
    if err != nil {
        return fmt.Errorf("ffmpeg failed: %w, output: %s", err,
string(output))
    }
    fmt.Printf("MP3 文件保存: %s\n", mp3Path)
    return nil
}

// PlayOggFile 播放 Ogg 文件 (使用 ffplay 或 VLC, 需本地安装)。
// 优先使用 ffplay (FFmpeg 自带, 轻量), fallback 到 VLC。
// 参数:
//   - oggPath: Ogg 文件路径。
// 返回: 错误 (如果播放失败)。
func PlayOggFile(oggPath string) error {
    // 尝试 ffplay (默认 FFmpeg 工具)
    cmd := exec.Command("ffplay", "-nodisp", "-autoexit", oggPath) // -
nodisp: 无视频窗口, -autoexit: 播放结束自动退出
    if err := cmd.Run(); err == nil {
        fmt.Printf("播放 Ogg 文件: %s (via ffplay)\n", oggPath)
        return nil
    }

    // Fallback 到 VLC
    cmd = exec.Command("vlc", "--intf", "dummy", "--play-and-exit",
oggPath) // dummy 接口无 GUI, play-and-exit 自动退出
    if err := cmd.Run(); err != nil {
        return fmt.Errorf("playback failed (ffplay/vlc): %w", err)
    }
}

```

```

}
fmt.Printf("播放 Ogg 文件: %s (via VLC)\n", oggPath)
return nil
}

// 示例主函数: 演示完整流程 (接收流 -> 保存 Ogg -> 可选转换 MP3 -> 播放
Ogg)。
func main() {
    // 配置 (可通过环境变量覆盖: STREAM_URL, OUTPUT_DIR, OUTPUT_NAME,
    FRAME_MS, OUTPUT_MP3)
    wsURL := os.Getenv("STREAM_URL")
    if wsURL == "" {
        wsURL = "ws://119.91.131.93:13008/stream/20090fbc-465c-4960-
825d-ffee5e6301ff?
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhY2NvdW50SWQiOiJlLCJleHBpcmVBdFRpbWUiOiJ1OTIwMDAwMDAwMDAwMDAwMCwiaXNzIjoic2lkY2hhaSIsImV
4cCI6I6MjAwMjE2MTU5NywibmJmIjoxNzQyOTYxNTk3LCJpYXQiOiJlE3NDI5NjE1OTd9.v2BmNT
3nq0na20IY40Ch_yL2BHZupYd7U1YmbtaD6c1" // 示例 URL, 替换为真实ws订阅地址
    }
    outDir := os.Getenv("OUTPUT_DIR")
    baseName := os.Getenv("OUTPUT_NAME")
    frameMS := 40
    if fm := os.Getenv("FRAME_MS"); fm != "" {
        fmt.Sscanf(fm, "%d", &frameMS)
    }

    // 步骤 1: 无限接收流并保存 Ogg (直到 WSS 断开)
    oggPath, err := StreamToOgg(wsURL, outDir, baseName, frameMS)
    if err != nil {
        fmt.Printf("StreamToOgg 失败: %v\n", err)
        return
    }

    // 步骤 2: 可选转换 MP3
    if os.Getenv("OUTPUT_MP3") == "1" {
        mp3Path := oggPath[:len(oggPath)-4] + ".mp3" // 替换 .ogg 扩展
        if err := ConvertToMP3(oggPath, mp3Path); err != nil {
            fmt.Printf("ConvertToMP3 警告: %v\n", err)
        }
    }
}

```

7. 限制说明

1. 连接限制

- a. appToken验证失败将直接断开连接

2. 数据限制

- a. 数据必须按顺序处理
- b. Base64编码的数据需要自行解码

3. 时间限制

- a. 音频流有效期有限
- b. 长时间无数据传输将自动断开连接

4.7 开发者信息更新

回调格式

ContentType: application/json

请求方式

POST

回调说明

1. 用于企业内开发者信息发生变化，包括设备绑定和解绑时回调第三方应用服务器地址
2. 管理员需在设备管理平台-环境配置“开发者信息”回调地址Url
3. 回调地址Url需支持POST协议

JSON数据包说明

参数名	类型	描述
callback	字符串	固定为 DEVELOPER_INFORM
data	开发者信息列表	开发者信息数据

<DEVELOPER_INFORM>说明：

参数名	类型	描述
eventId	string	本次事件唯一 ID
timestamp	int64	事件发生时间戳（秒）
developerId	string	开发者账户ID
developerName	string	开发者名称
phone	string	绑定手机号
recordingEncryption	bool	录音文件是否加密（0关闭/1开启）
recordingExpireDays	int	录音有效期（天）
recordingFileRetentionHours	int	录音文件链接有效期（小时）
changedFields	string[]	本次哪些开发者信息字段发生了变更
deviceAction	string	"bind"绑定 或 "unbind"解绑
deviceIds	string[]	本次绑定/解绑的设备ID列表

operateTime	int64	本次批量操作的时间戳 (有设备变更时必返回)
-------------	-------	---------------------------

JSON数据包示例

复制代码

```
{
  "eventId": "evt_2025112112345678901234567890",
  "timestamp": 1734964800,
  "developerId": "1234",
  "developerIdName": "张三",
  "phone": "18907124201",
  "recordingEncryption": true,
  "recordingExpireDays": 30,
  "recordingFileRetentionHours": 24,
  "changedFields": [
    "developerIdName",
    "phone",
    "recordingSecret"
  ],
  "deviceAction": "bind",
  "deviceIds": [
    "dev_003456789abc",
    "dev_003456789def",
    "dev_003456789ghi",
    "dev_003456789jkl"
  ],
  "operateTime": 1734964000,
}
```